

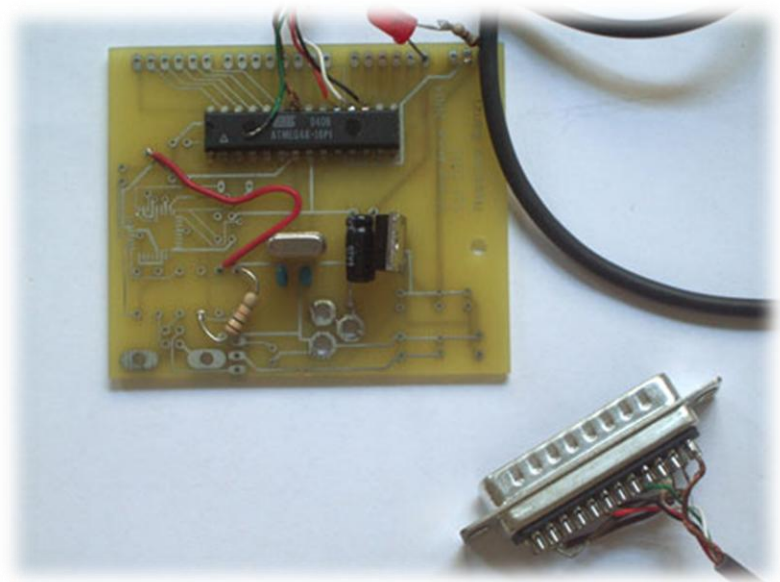


Chương 7: Arduino



Arduino history...

- ❑ Bắt đầu là một dự án nhằm giúp mọi người (từ nghệ sỹ, người chơi nghiệp dư,... tới sinh viên chuyên ngành) tiếp cận smart devices/IoT dễ hơn
- ❑ Hiện tại: board lập trình nổi tiếng nhất trong giới DIY!
- ❑ arduino.cc vs org



<http://spectrum.ieee.org/geek-life/hands-on/the-making-of-arduino>

Ưu điểm Arduino?

- ❑ Thiết kế mở, chuẩn hóa cả phần cứng và phần mềm.
- ❑ Cân bằng giữa tính năng và tính dễ sử dụng.
 - | Ngôn ngữ C++ đơn giản hóa.
- ❑ Cộng đồng người dùng rất lớn.
- ❑ Sản phẩm rất đa dạng, nhiều tính năng.
- ❑ Rẻ!

Arduino hardware

□ Nano



Arduino Nano 33 IoT



Arduino Nano RP2040
Connect



Arduino Nano 33 BLE Sense

□ MKR



Arduino MKR WAN 1300



Arduino MKR WAN 1310



Arduino MKR GSM 1400

□ Classic



Arduino UNO R3



Arduino Mega 2560
Rev3



Arduino Leonardo

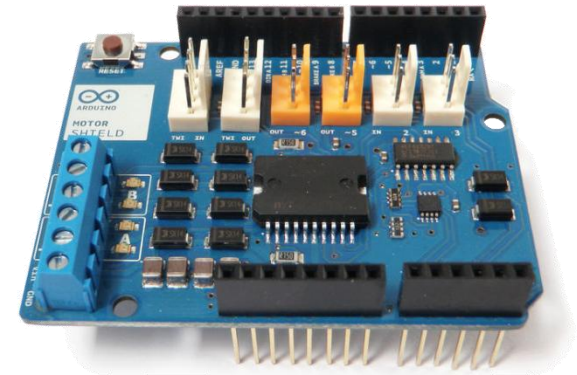


Arduino UNO Mini
Limited Edition

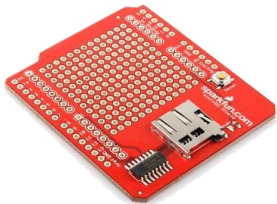
□ Arduino PLC

Shields

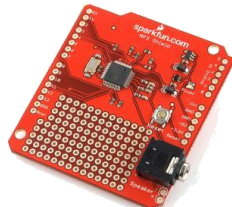
- ❑ Cắm lên trên mạch Arduino.
- ❑ Cung cấp chức năng mở rộng
 - | Ethernet
 - | GPS
 - | Motor
 - | Prototype
 - | ...
- ❑ shieldlist.org



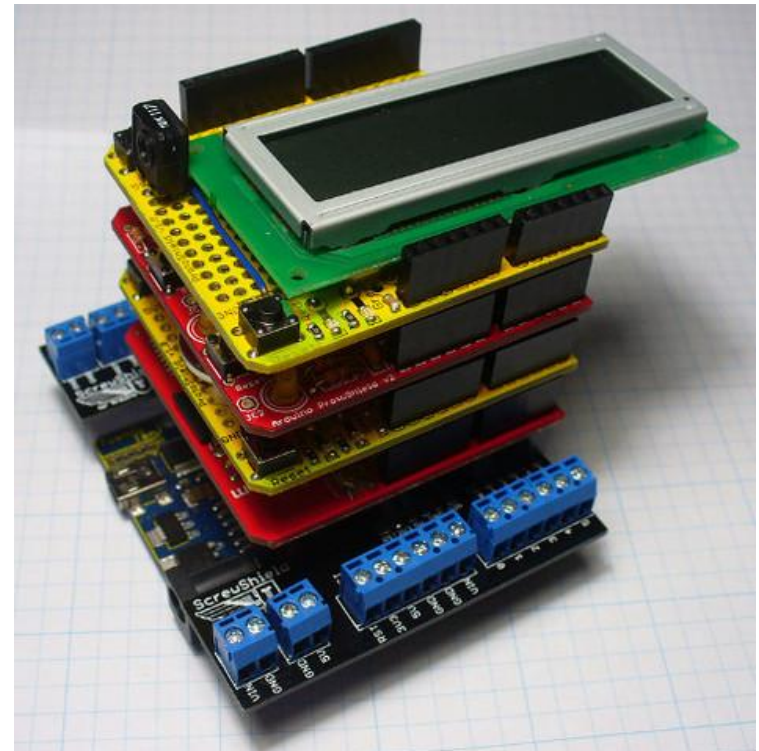
Micro SD



MP3 Trigger



LCD

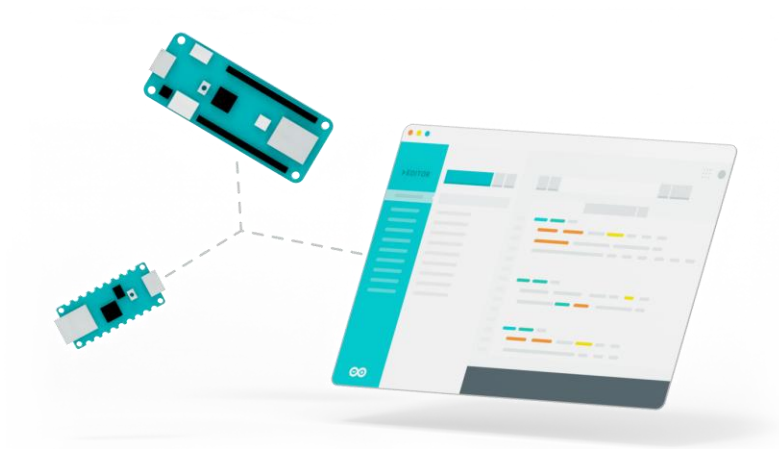


Arduino software

- ❑ Desktop IDE:
 - | Arduino IDE 2.x/1.8
 - | Arduino PLC IDE
- ❑ Cloud-based IDE
- ❑ 3rd party tools: TinkerCad online

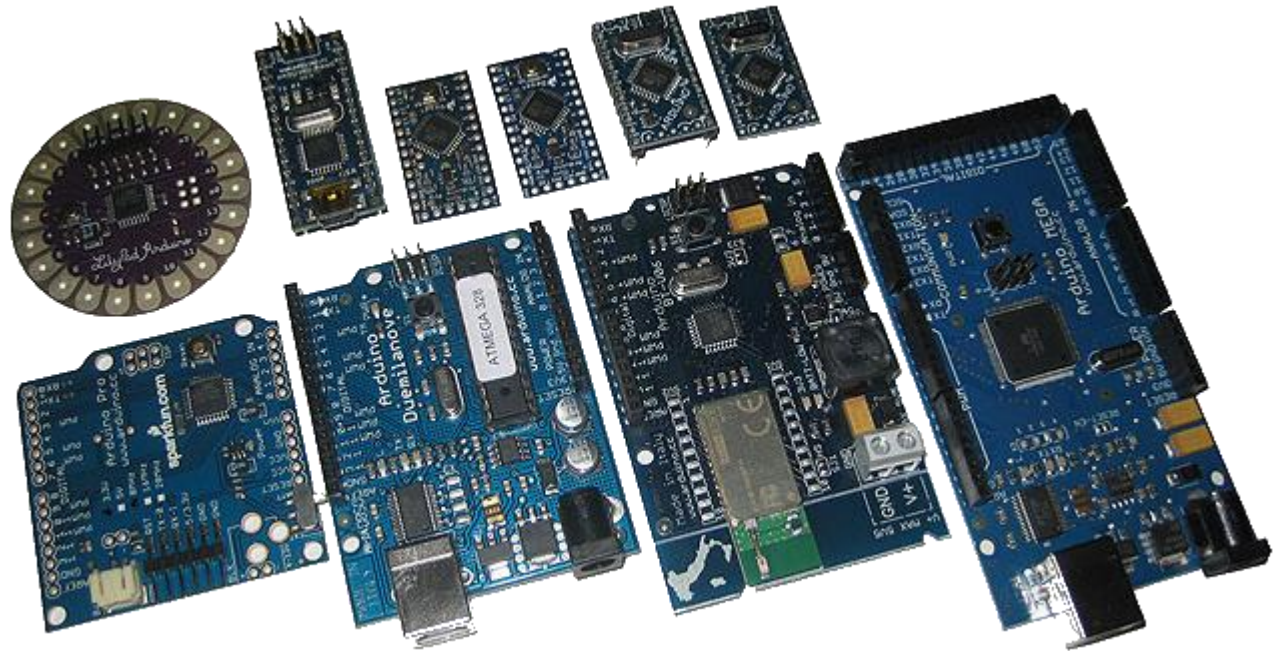
Arduino cloud

- ❑ Platform cho các hệ thống IoT, home automation,...



Arduino boards

- ❑ Leonardo
- ❑ Due
- ❑ Micro
- ❑ LilyPad
- ❑ Esplora
- ❑ Uno



Arduino - Products

Uno

- ❑ ATmega328P
- ❑ 14 digital I/O
- ❑ 6 analog input
- ❑ 16 MHz clock
- ❑ 32 KB flash
- ❑ 2 KB SRAM

POWER
5V / 3.3V / GND

**Analog
INPUTS**

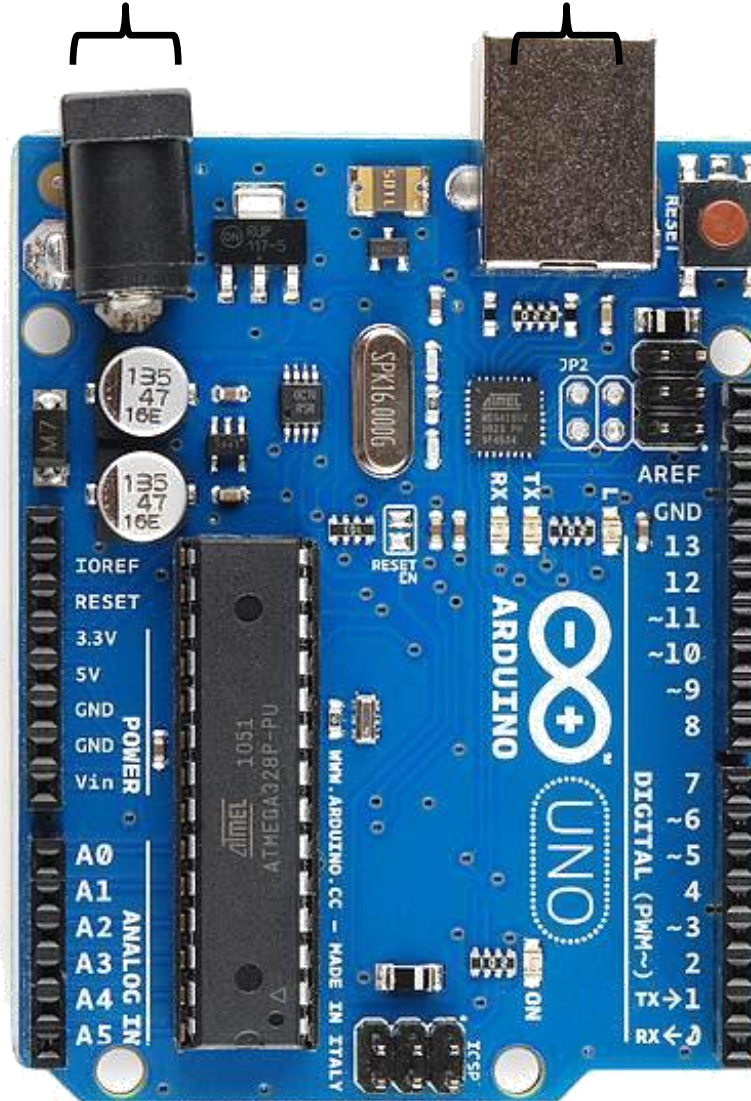
PWR IN

USB (to Computer)

RESET

SCL\SDA
(I2C Bus)

Digital I/O
PWM(3, 5, 6, 9, 10, 11)



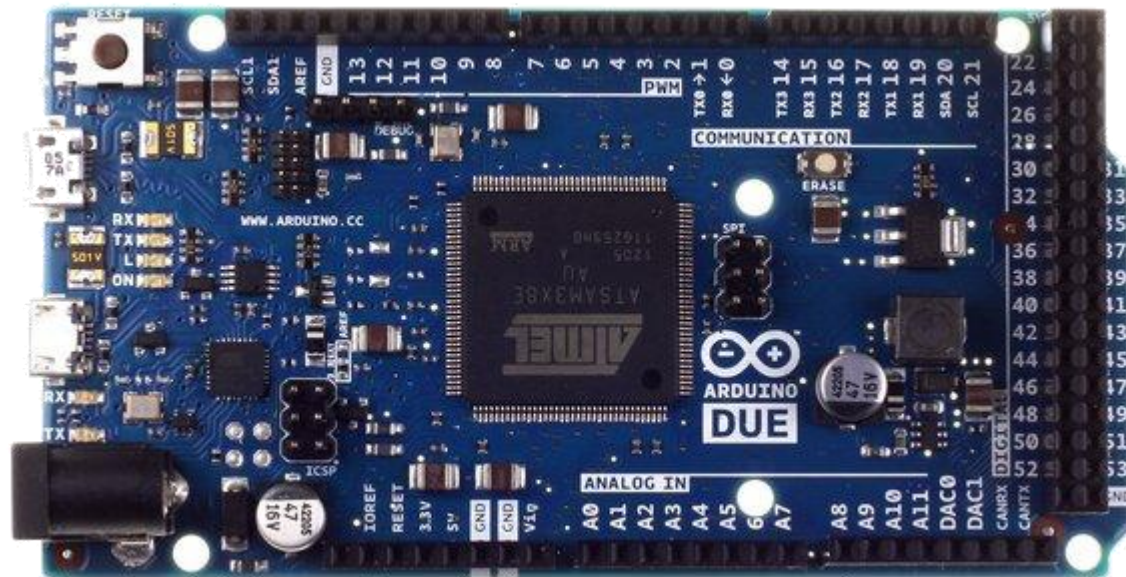
Arduino Mega 2560

- ❑ Nâng cấp của Uno, rất phổ biến
 - | Nhiều chân: 54 digital I/O, 16 analog input.
 - | Nhiều tài nguyên hơn: 256 KB flash, 8 KB RAM, 4 UART,...



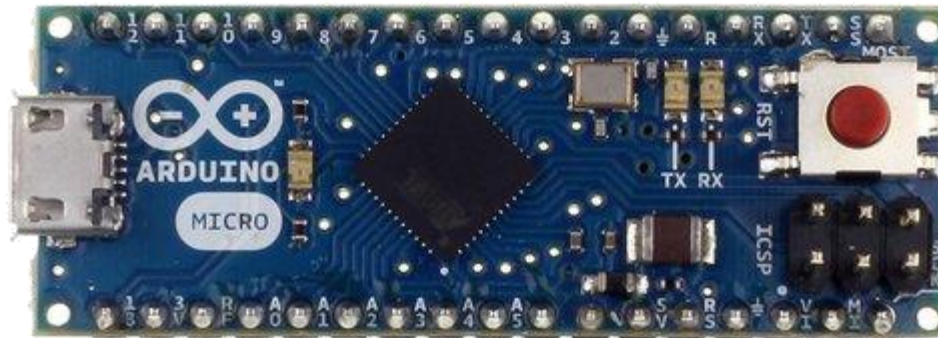
Due

- ❑ Giống Mega 2560
- ❑ Chip ARM, tốc độ cao (84 MHz)
- ❑ 512 KB flash, 96 KB RAM, nhiều ngoại vi
- ❑ VCC 3.3V



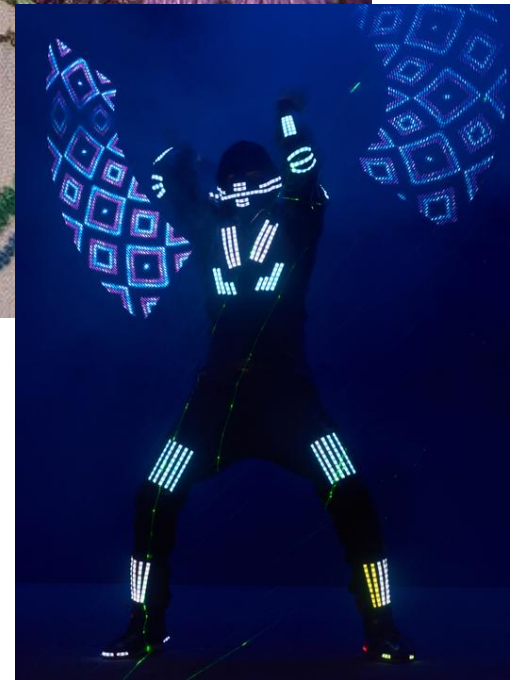
Micro

- ❑ Thuộc dòng board kích thước nhỏ: Micro, Nano, Mini
- ❑ Có thể cắm trên breadboard
- ❑ 32 KB flash, 2.5 KB RAM, 16 MHz clock



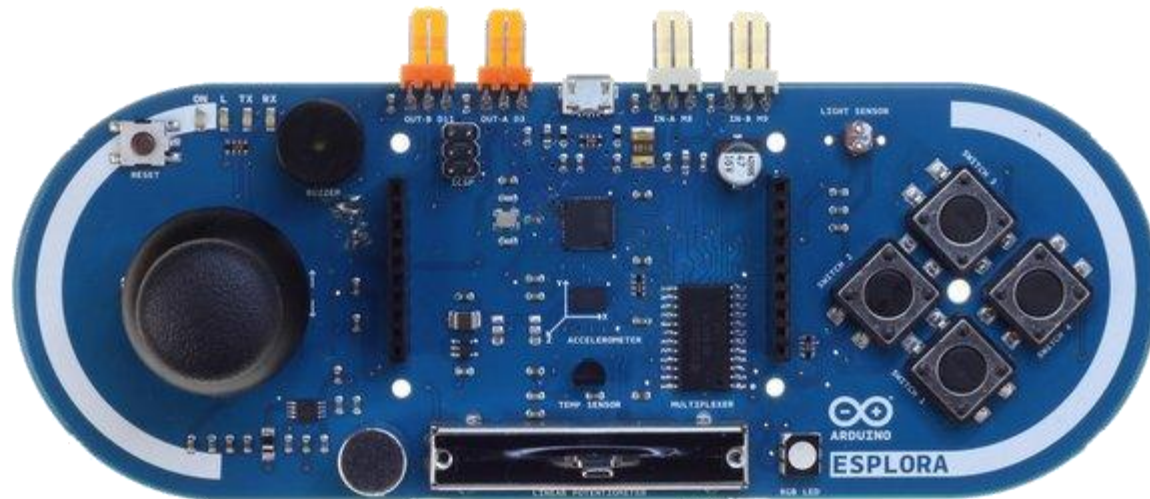
LilyPad (retired)

- ❑ Dành cho ứng dụng wearable, e-textile



Esplora (retired)

- ❑ Game controller
- ❑ Có joystick, four buttons, linear potentiometer (slider), microphone, light sensor, temperature sensor, three-axis accelerometer.



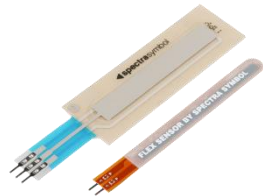
Cảm biến

❑ Temperature

❑ Light

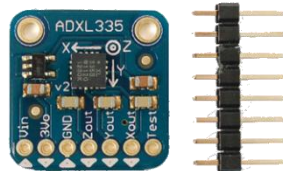


❑ Angle



❑ Switches

❑ Accelerometer



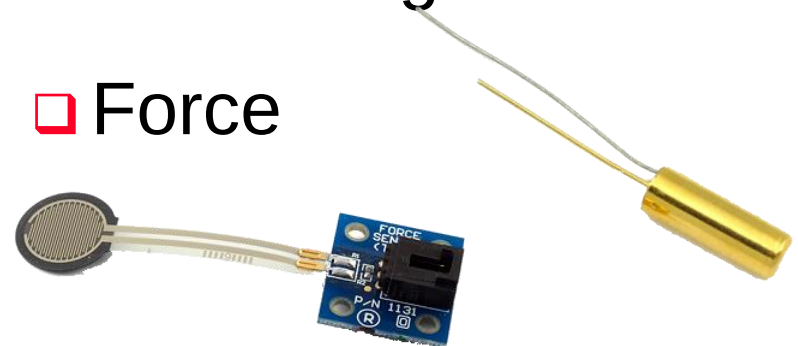
❑ Infrared sensor & light

❑ Hall effect sensor and magnet



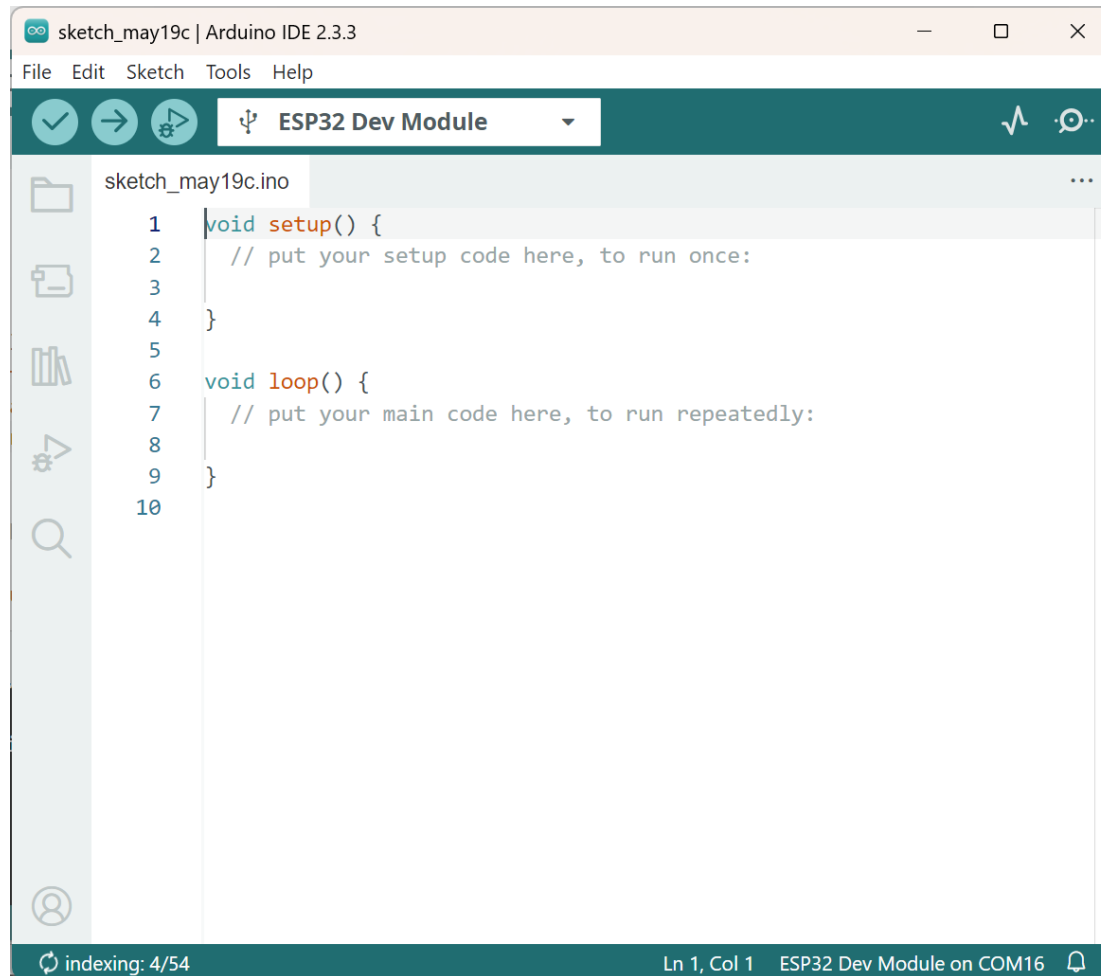
❑ Ball tilt sensor (for measuring orientation)

❑ Force



Arduino IDE 2.x

❑ 2 hàm chính: setup() và loop()

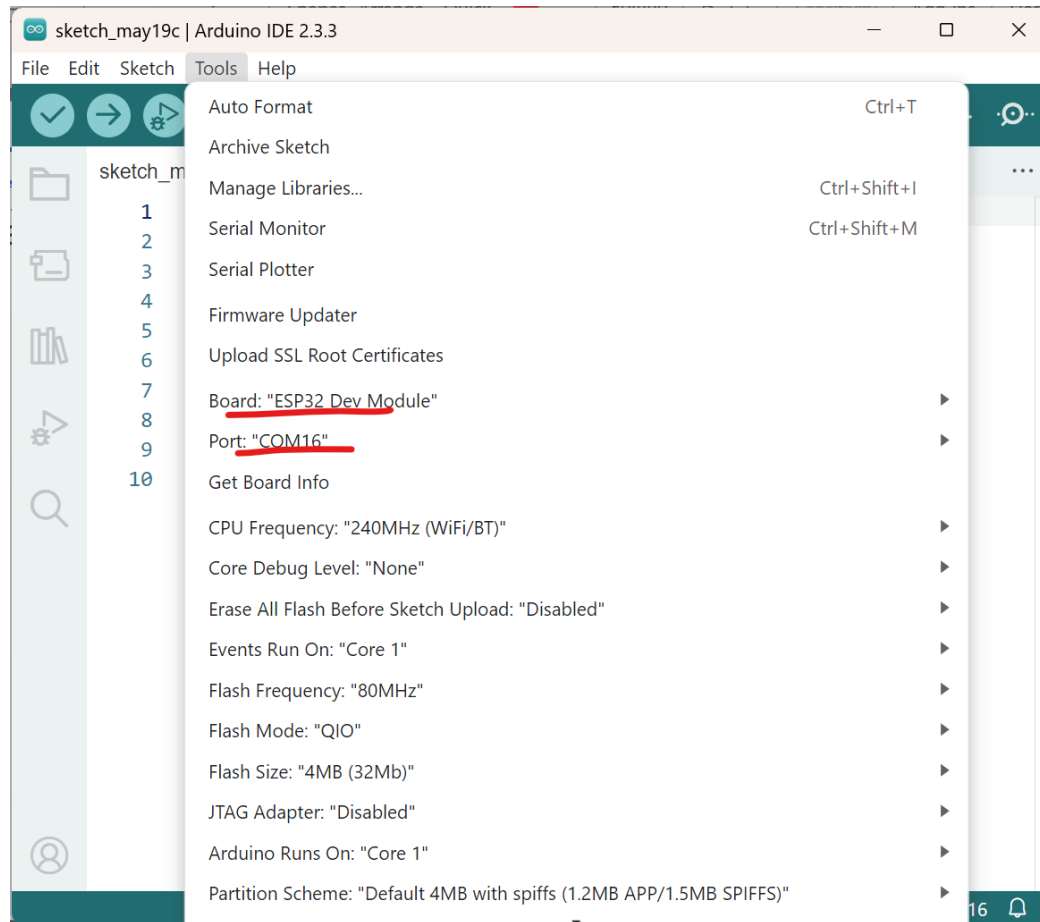


```
sketch_may19c.ino
1 void setup() {
2   // put your setup code here, to run once:
3
4 }
5
6 void loop() {
7   // put your main code here, to run repeatedly:
8
9 }
10
```

indexing: 4/54 Ln 1, Col 1 ESP32 Dev Module on COM16

Settings: Tools → Serial Port

- ❑ Chọn board và port (phải chọn đúng thì mới nạp code và chạy được)



Arduino programming

- ❑ Ngôn ngữ: dựa trên cú pháp của C++.
- ❑ Programs được gọi là 'sketches'.
- ❑ Mỗi sketch có 2 hàm bắt buộc:
 - | void setup()
 - | void loop()
- ❑ setup() chạy một lần khi khởi tạo/reset mạch.
- ❑ loop() chạy lặp vô hạn.

Lập trình với Arduino

❑ Ghép nối tín hiệu số

- | `pinMode(pin, mode)`
Đặt chế độ vào/ra cho chân cổng
- | `digitalWrite(pin, value)`
Ghi dữ liệu ra chân cổng
- | `digitalRead(pin)`
Đọc giá trị từ chân cổng

```
pinMode(13, OUTPUT);  
digitalWrite(13, HIGH);
```

Lập trình với Arduino

❑ Các hàm thời gian

- | `delay()`: Tạo trễ theo milli giây
- | `delayMicroseconds()`: Tạo trễ micro giây

❑ `micros()`

- | Lấy thời gian hiện tại (từ lúc mạch chạy) theo micro giây

❑ `millis()`

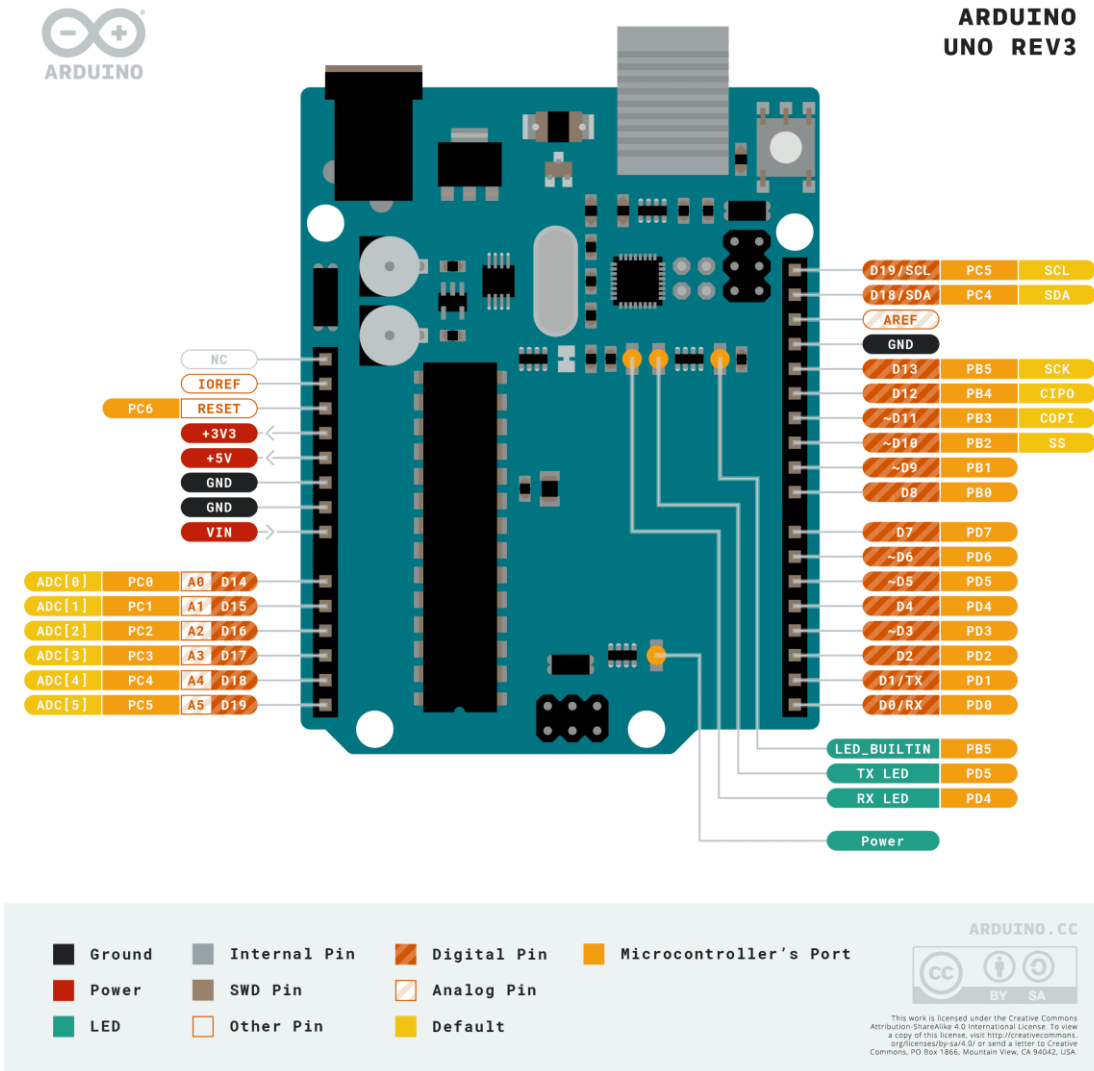
- | Lấy thời gian hiện tại (từ lúc mạch chạy) theo milli giây

❑ **Chú ý:**

- | **Nên tránh dùng hàm `delay()`**
- | **Các hàm delay đều không cấm ngắt**

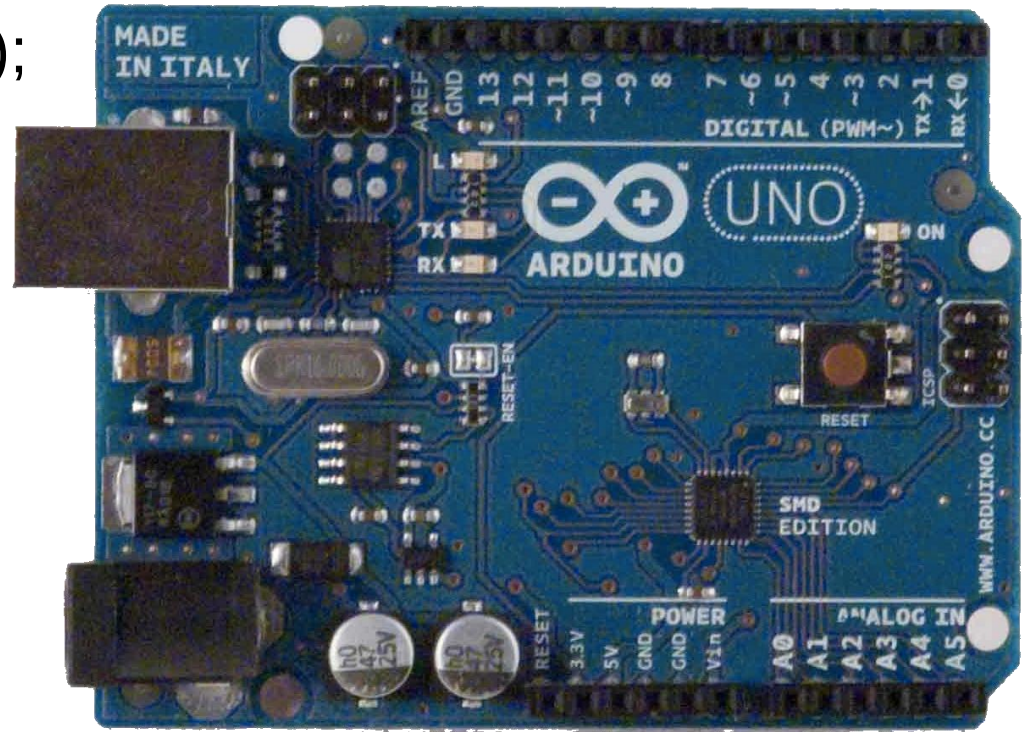
Arduino pinout

Uno R3



VD: Blink Sketch

```
void setup( ) {  
    pinMode(13, OUTPUT);  
}  
void loop( ) {  
    digitalWrite(13, HIGH);  
    delay(1000);  
    digitalWrite(13, LOW);  
    delay(1000);  
}
```



So sánh

❑ STM32F4

```
#include <...>
void MX_GPIO_Init();
void main() {
    ...
    MX_GPIO_Init();
    ...
    while (1) {
        HAL_GPIO_WritePin(...);
        HAL_Delay(1000);
        HAL_GPIO_WritePin(...);
        HAL_Delay(1000);
    }
}
```

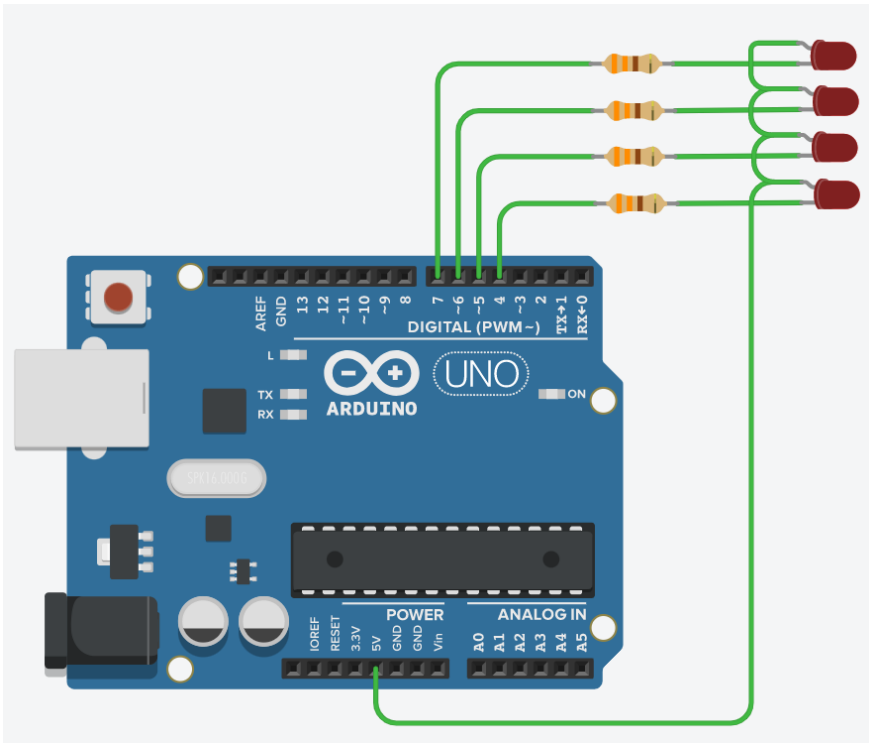
❑ Arduino C *(đơn giản hơn)*

```
void setup() {
    pinMode(13, OUTPUT);
}

void loop() {
    digitalWrite(13, HIGH);
    delay(1000);
    digitalWrite(13, LOW);
    delay(1000);
}
```


Ví dụ: LED Blink Sketch (TinkerCad)

📄 <https://www.tinkercad.com/>



```
int LED1 = 7;  
int LED2 = 6;  
int LED3 = 5;  
int LED4 = 4;
```

```
void setup()  
{  
  pinMode(LED_BUILTIN, OUTPUT);  
  pinMode(LED1, OUTPUT);  
  pinMode(LED2, OUTPUT);  
  pinMode(LED3, OUTPUT);  
  pinMode(LED4, OUTPUT);  
}
```

```
void loop()  
{  
  digitalWrite(LED_BUILTIN, HIGH);  
  digitalWrite(LED1, HIGH);  
  digitalWrite(LED2, HIGH);  
  digitalWrite(LED3, LOW);  
  digitalWrite(LED4, LOW);  
  delay(1000); // Wait for 1000 millisecond(s)  
  digitalWrite(LED_BUILTIN, LOW);  
  digitalWrite(LED1, LOW);  
  digitalWrite(LED2, LOW);  
  digitalWrite(LED3, HIGH);  
  digitalWrite(LED4, HIGH);  
  delay(1000); // Wait for 1000 millisecond(s)  
}
```

Nháy LED không dùng delay

```
const int ledPin = LED_BUILTIN;
int ledState = LOW;
unsigned long previousMillis = 0;
const long interval = 1000;

void setup() {
  pinMode(ledPin, OUTPUT);
}

void loop() {
  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval)
  {
    previousMillis = currentMillis;
    if (ledState == LOW) {
      ledState = HIGH;
    } else {
      ledState = LOW;
    }
    digitalWrite(ledPin, ledState);
  }
}
```

Lập trình với Arduino

❑ Làm việc với ngắt ngoài

BOARD	DIGITAL PINS USABLE FOR INTERRUPTS
Uno, Nano, Mini, other 328-based	2, 3
Uno WiFi Rev.2, Nano Every	all digital pins
Mega, Mega2560, MegaADK	2, 3, 18, 19, 20, 21
Micro, Leonardo, other 32u4-based	0, 1, 2, 3, 7
Zero	all digital pins, except 4
MKR Family boards	0, 1, 4, 5, 6, 7, 8, 9, A1, A2
Nano 33 IoT	2, 3, 9, 10, 11, 13, 15, A5, A7
Nano 33 BLE, Nano 33 BLE Sense	all pins
Due	all digital pins
101	all digital pins (Only pins 2, 5, 7, 8, 10, 11, 12, 13 work with CHANGE)

Lập trình với Arduino

❑ Làm việc với ngắt ngoài

- | `attachInterrupt()`
- | `detachInterrupt()`
- | `digitalPinToInterrupt()`
- | `interrupts()`
- | `noInterrupts()`

❑ Các chế độ ngắt

- | `LOW`
- | `CHANGE`
- | `FALLING`
- | `RISING`

```
const byte ledPin = 13;  
const byte interruptPin = 2;  
volatile byte state = LOW;
```

```
void setup() {  
    pinMode(ledPin, OUTPUT);  
    pinMode(interruptPin,  
    INPUT_PULLUP);
```

```
    attachInterrupt(digitalPinToInterrupt(  
        interruptPin),  
        blink, CHANGE);  
}
```

```
void loop() {  
    digitalWrite(ledPin, state);  
}
```

```
void blink() {  
    state = !state;  
}
```

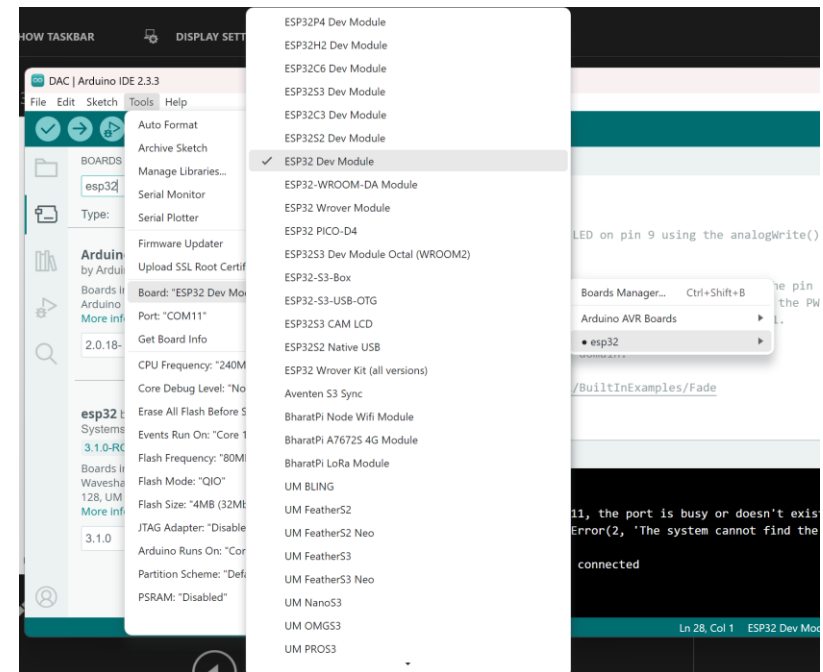
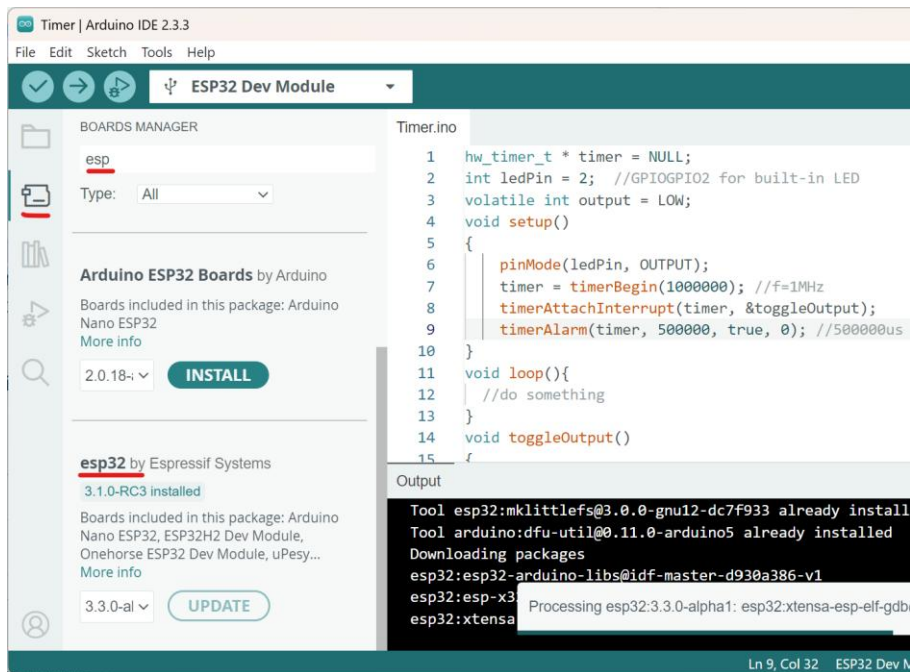
Lập trình với ESP32 DEV KIT V1

❑ Cài đặt Arduino IDE

- File → Preferences: *Additional Board Manager URLs*

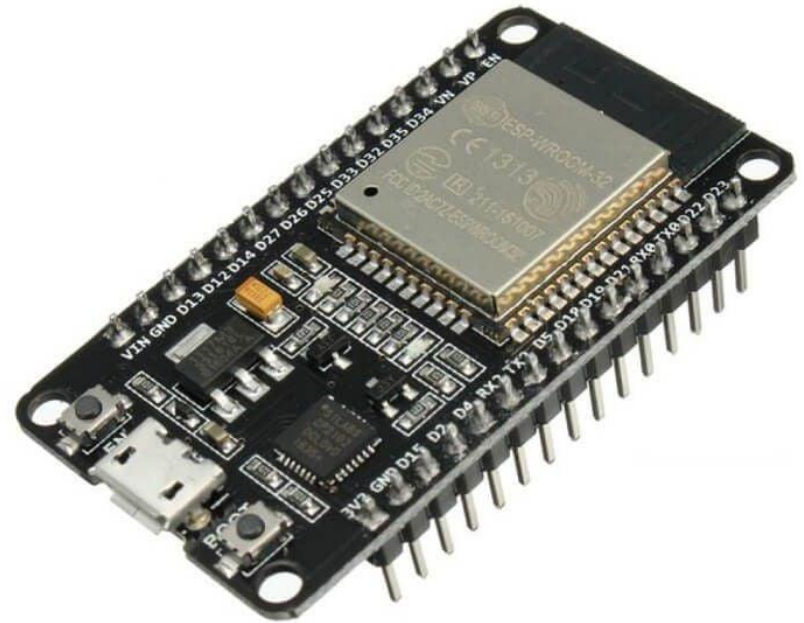
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

- Cài đặt package cho ESP32: Tools → Board → Board Manager

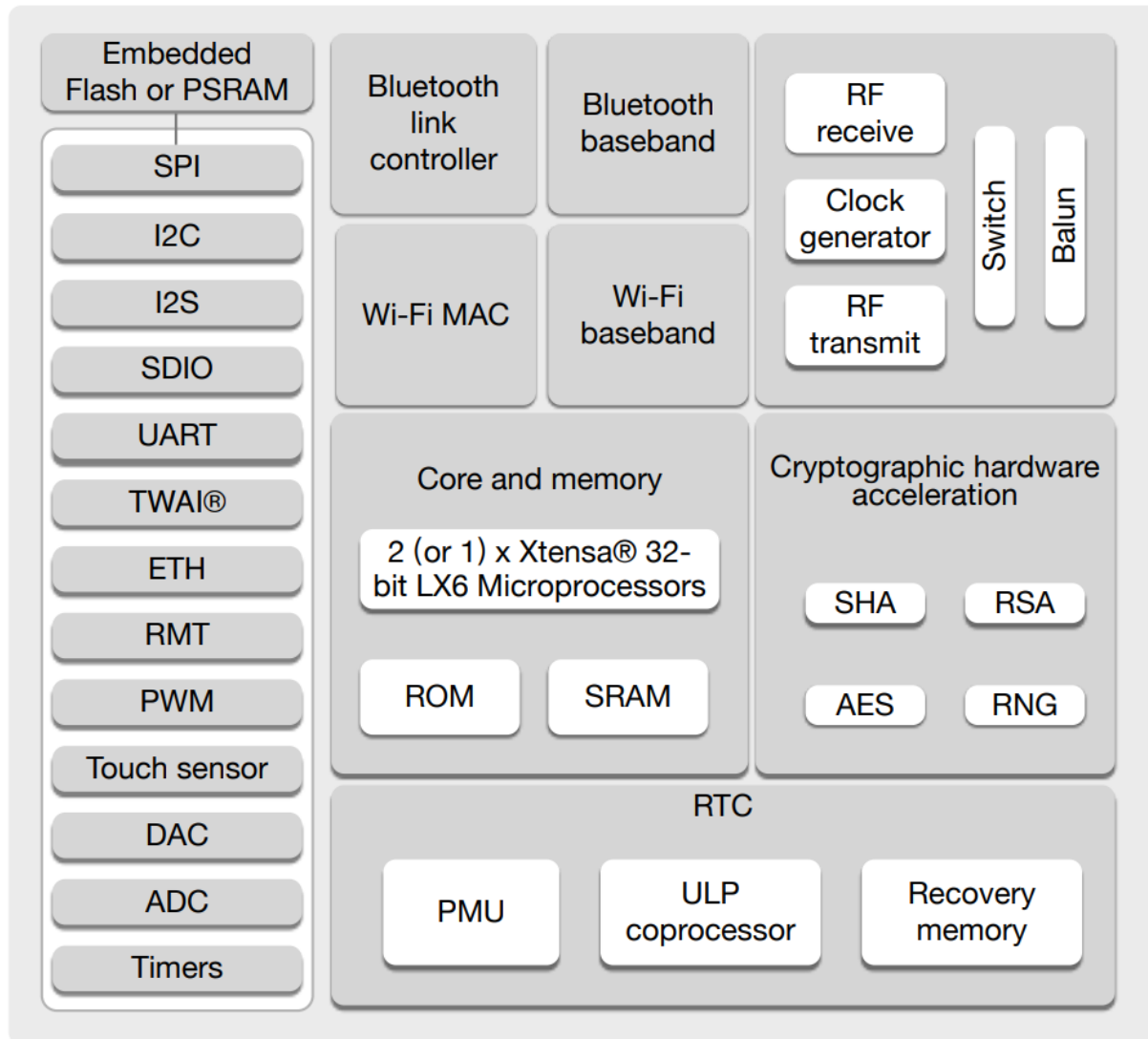


ESP32 DEV KIT V1

- ❑ Main processor:
 - | Xtensa dual-core (or single-core) 32-bit LX6
 - | Frequency: 160 or 240 MHz
 - | Performance: ~600 DMIPS
- ❑ Ultra low power co-processor
- ❑ Memory:
 - | 448 KiB ROM
 - | 520 KiB SRAM
- ❑ Wireless connectivity:
 - | Wi-Fi: 802.11 b/g/n
 - | Bluetooth: v4.2 BR/EDR and BLE (shares the radio with Wi-Fi)

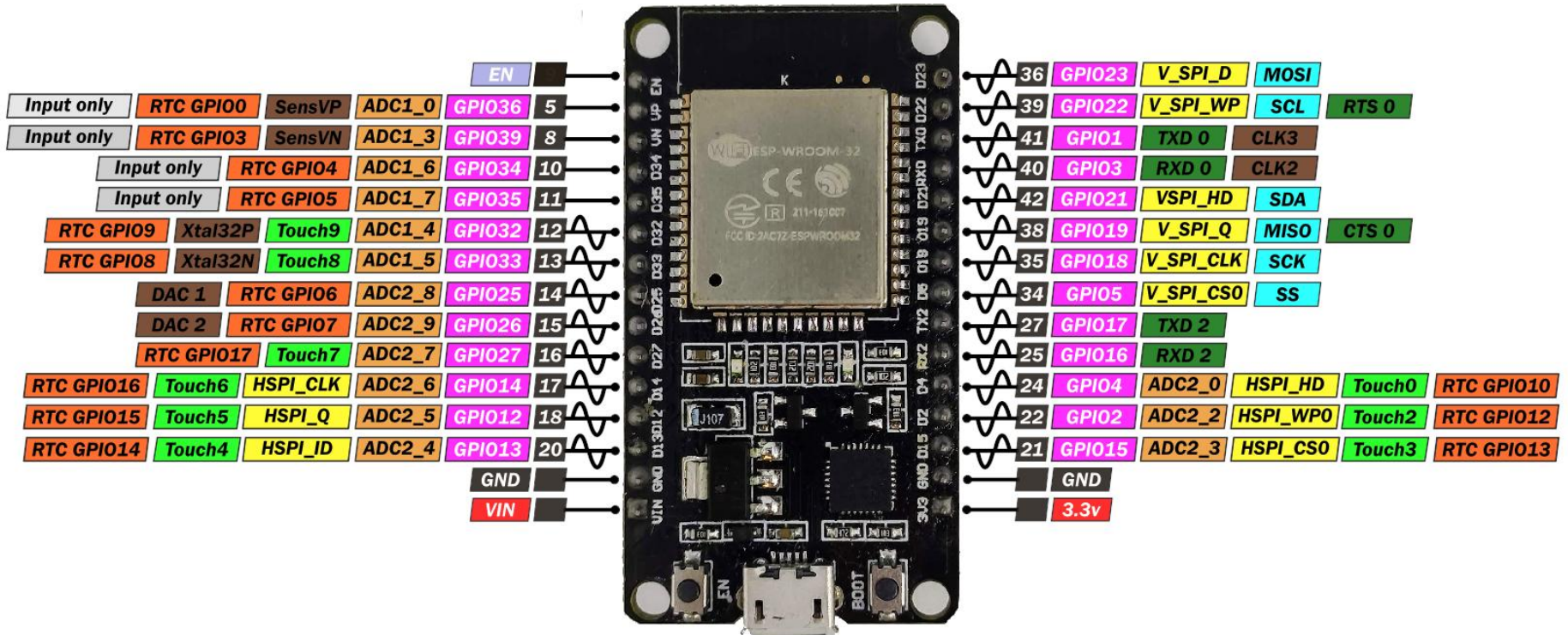


ESP32 Function Block Diagram



Pinout

ESP32 DEV KIT V1 PINOUT



Ví dụ 1: LED Blinking

❑ Yêu cầu:

- | Nối 1 LED đến GPIO33 của KIT (nối xuống GND qua điện trở 330 ohm)
- | Biên dịch chương trình, nạp vào KIT, và quan sát LED.

```
void setup() {  
    pinMode(33, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(33, HIGH);  
    delay(1000);  
    digitalWrite(33, LOW);  
    delay(1000);  
}
```

Ví dụ 2: LED Blinking

❑ Yêu cầu:

- | Không sử dụng hàm `delay()` để tạo hiệu ứng. Tại sao?

```
int ledState = LOW;
unsigned long previousMillis = 0;
const long interval = 500;
void setup() {
    pinMode(33, OUTPUT); //GPIO33
}
void loop() {
    unsigned long currentMillis = millis();
    if (currentMillis - previousMillis >= interval) {
        previousMillis = currentMillis;
        if (ledState == LOW) {
            ledState = HIGH;
        } else {
            ledState = LOW;
        }
        digitalWrite(33, ledState);
    }
}
```

Ví dụ 3: External Interrupt

- ❑ Tất cả các GPIO pins của ESP32 có thể dùng để phục vụ ngắt ngoài.
- ❑ Yêu cầu:
 - | Cấu hình GPIO 26 để nhận interrupt request (falling trigger). Sử dụng nút bấm.
 - | Nối 1 LED đến GPIO 33.
 - | Biên dịch chương trình, nạp vào KIT. Bấm nút và quan sát LED.

Ví dụ 3: External Interrupt

```
const byte interruptPin = 26;    //GPIO26
const byte ledPin = 33;         //GPIO33
volatile byte state = LOW;
void setup() {
    pinMode(ledPin, OUTPUT);
    pinMode(interruptPin, INPUT_PULLUP); // INPUT_PULLUP vs INPUT?
    attachInterrupt(digitalPinToInterrupt(interruptPin), blink, FALLING);
}
void loop() {
    digitalWrite(ledPin, state);
}
void blink() {
    state = !state;
}
```


Ví dụ 4: Timer Interrupt

- ❑ 4 general-purpose timers (dựa vào 16-bit prescaler và 64-bit auto-reload-capable up/down counter)
- ❑ 3 watchdog timers
- ❑ Yêu cầu:
 - | Nối 1 LED đến GPIO 33.
 - | Biên dịch chương trình, nạp vào KIT, và quan sát LED.

Ví dụ 4: Timer Interrupt

```
hw_timer_t * timer = NULL;
int ledPin = 2; //GPIO2 for built-in LED, có thể thay bằng 33
volatile int output = LOW;
void setup()
{
    pinMode(ledPin, OUTPUT);
    timer = timerBegin(1000000); //f=1MHz
    timerAttachInterrupt(timer, &toggleOutput);
    timerAlarm(timer, 500000, true, 0); //500000us cycle
}
void loop(){
    //do something
}
void toggleOutput()
{
    digitalWrite(ledPin, output);
    output = !output;
}
```

Arduino-ESP32 Timer API version 3.x

❑ timerBegin: khởi tạo timer

- | `hw_timer_t * timerBegin(uint32_t frequency);`
 - frequency: tần số đếm của timer.

❑ timerAttachInterrupt:

- | `void timerAttachInterrupt(hw_timer_t * timer, void (*userFunc)(void));`
 - timer: con trỏ tới timer đã khởi tạo.
 - userFunc: con trỏ tới hàm xử lý ngắt.

❑ timerAlarm:

- | `void timerAlarm(hw_timer_t * timer, uint64_t alarm_value, bool autoreload, uint64_t reload_count);`
 - timer: con trỏ tới timer đã khởi tạo.
 - alarm_value: giá trị bộ đếm khi gây ngắt.
 - autoreload: enabled/disabled tự nạp lại giá trị bộ đếm của timer.
 - reload_count: số lần tự nạp lại bộ đếm (0 = unlimited). Không có tác dụng nếu autoreload = false.

Analog output và Digital to Analog Converter (DAC)

❑ Có 2 chế độ analog output

- | `analogWrite()`: xuất tín hiệu PWM ra chân chip
 - Giả analog, sử dụng công suất trung bình của tín hiệu.
- | `dacWrite()`: sử dụng bộ DAC có sẵn trên chip để xuất điện áp có giá trị tương ứng ra chân chip.
 - ESP32 WROOM dev kit có 2 chân DAC: GPIO25 và GPIO26
 - Tín hiệu đầu ra là analog thật!

❑ Ví dụ 5:

- | Xuất tín hiệu PWM ra D2 để điều khiển độ sáng của đèn LED.
- | Xuất tín hiệu điện áp biến thiên giữa 0 và 3.3V ra chân DAC1 và DAC2.

Ví dụ 5: Xuất tín hiệu analog

```
#define DAC1 25 //Chạy ví dụ ADC_Graphics trên STM32F429, nối chân 25 hoặc 26 của ESP32
#define DAC2 26 //vào PC3 của STM32F429 sẽ thấy quả bóng đỏ di chuyển giữa 2 cạnh

int led = 2;          // the PWM pin the LED is attached to
int brightness = 0;   // how bright the LED is
int fadeAmount = 1;   // how many points to fade the LED by

// the setup routine runs once when you press reset:
void setup() {
    pinMode(led, OUTPUT);
}

// the loop routine runs over and over again forever:
void loop() {
    //set the brightness of LED using PWM:
    analogWrite(led, brightness);

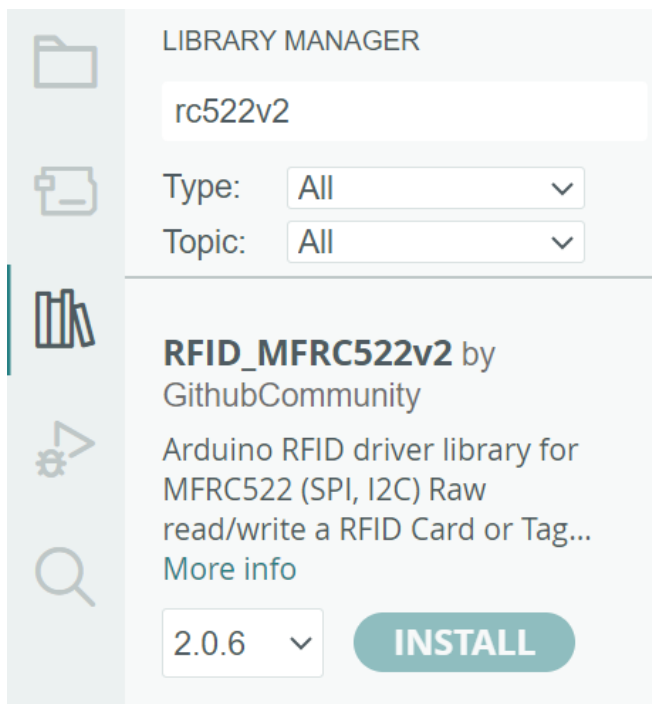
    //output to DAC pins using Digital to Analog hardware
    dacWrite(DAC1, brightness);
    dacWrite(DAC2, 255-brightness);

    // change the brightness for next time through the loop:
    brightness = brightness + fadeAmount;

    // reverse the direction of the fading at the ends of the fade:
    if (brightness <= 0 || brightness >= 255) {
        fadeAmount = -fadeAmount;
    }
    // wait for 2 milliseconds to see the dimming effect
    delay(2);
}
```

Ví dụ 6: SPI

- ❑ Ghép nối RC522 với ESP32
- ❑ Cài đặt thêm thư viện RC522v2 (trong Library Manager)
- ❑ Nối RC522 với ESP32 theo sơ đồ chân



ESP32

Signal	MFRC522	WROOM-32
RST/Reset	RST	21
SPI SS	SDA	5
SPI MOSI	MOSI	23
SPI MISO	MISO	19
SPI SCK	SCK	18

[OSSLibraries/Arduino_MFRC522v2](https://osslibraries.com/Arduino_MFRC522v2)

MFRC522v2 sample code

```
#include <MFRC522v2.h>
#include <MFRC522DriverSPI.h>
#include <MFRC522DriverPinSimple.h>
#include <MFRC522Debug.h>

MFRC522DriverPinSimple ss_pin(5); // Set Slave Select to D5.
MFRC522DriverSPI driver{ss_pin}; // Create SPI driver.
MFRC522 mfrc522{driver}; // Create MFRC522 instance.

void setup() {
    Serial.begin(115200);
    mfrc522.PCD_Init(); // Init MFRC522 board.
    MFRC522Debug::PCD_DumpVersionToSerial(mfrc522, Serial);
}

void loop() {
    // Reset the loop if no new card present on the sensor/reader.
    if (!mfrc522.PICC_IsNewCardPresent()) {
        return;
    }

    // Select one of the cards.
    if (!mfrc522.PICC_ReadCardSerial()) {
        return;
    }

    // Dump debug info about the card
    MFRC522Debug::PICC_DumpToSerial(mfrc522, Serial, &(mfrc522.uid));
}
```